

Reviewer Pack — Riemann Hypothesis and Circuit Entanglement

Entry 9112f3fb47e3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 16:05:13 UTC

Riemann Hypothesis and Circuit Entanglement

Entry ID: 9112f3fb47e3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 16:05:13 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Riemann Zeta Function) **Field B** (complexity object): Quantum Information Theory (Circuit Entanglement)

Statement:

The number of entangled qubits in a quantum circuit representing an n -bit Boolean function is bounded by the inverse Riemann zeta function evaluated at $s = 1/2 + i\tau$, where τ is a complex number associated with the function's Fourier transform.

Rationale (proposer's reasoning):

The Riemann zeta function encodes deep structures in number theory, and its evaluation for non-integer values has connections to quantum entanglement. This conjecture proposes a bridge between these two fields by linking the Riemann zeta function to the entanglement of Boolean functions when represented as quantum circuits.

Taxonomy category: ENTANGLEMENT-bound-by-RIEMANN-ZETA (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ba0632a2b644179f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all n -bit Boolean functions, the maximum number of entangled qubits in a corresponding quantum circuit is within ± 3 qubits of the inverse Riemann zeta function value at $s = 1/2 + i\tau$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def fourier_transform(f, n):
        result = [0] * (2 ** n)
        for k in range(2 ** n):
            sum_real = 0
            sum_imag = 0
            for j in range(2 ** n):
                angle = 2 * math.pi * k * j / (2 ** n)
                real_part = f(j) * math.cos(angle)
                imag_part = -f(j) * math.sin(angle)
                sum_real += real_part
                sum_imag += imag_part
            result[k] = (sum_real, sum_imag)
        return result

    def tau(fourier):
        n = len(fourier)
        sum_real = 0
        sum_imag = 0
        for k in range(n):
            sum_real += abs(fourier[k][0])
            sum_imag += abs(fourier[k][1])
        return Fraction(sum_real, n), Fraction(sum_imag, n)

    def max_entangled_qubits(fourier):
        n = len(fourier)
        max_qubits = 0
        for k in range(n):
            if fourier[k][0] != 0 or fourier[k][1] != 0:
                max_qubits += 1
        return max_qubits

    def riemann_zeta(s, tol=1e-10):
```

```

    result = 0
    n = 1
    while True:
        term = Fraction(1, n ** s)
        if abs(term) < tol:
            break
        result += term
        n += 1
    return result

def riemann_zeta_inv(s):
    return 1 / riemann_zeta(s)

def random_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2 ** n)]

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    f = random_boolean_function(n)
    fourier = fourier_transform(f, n)
    tau_real, tau_imag = tau(fourier)
    expected_bound = abs(riemann_zeta_inv(Fraction(1, 2) + Fraction(tau_real, tau_imag)))
    actual_value = max_entangled_qubits(fourier)

    if actual_value < expected_bound - 3 or actual_value > expected_bound + 3:
        conjecture_holds = False
        counterexample = f"n={n}, expected_bound={expected_bound}, actual_value={actual_value}"
        break

    total_metric_value += actual_value
    instances_tested += len(fourier)
    n_max = max(n_max, n)

return {
    "metric_name": "max_entangled_qubits",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results)) ** 0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff9cf9dc.py", line 107, in <module>
 result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff9cf9dc.py", line 78, in run_trial

fourier = fourier_transform(f, n)

^^^^^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff9cf9dc.py", line 28, in fourier_transform

real_part = f(j) * math.cos(angle)

^^^^

TypeError: 'list' object is not callable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the error in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13588
2	preregistration	ollama_remote	glm4:latest	0	0	10310
3	novelty	ollama_remote	glm4:latest	0	0	8735
4	novelty	ollama_remote	glm4:latest	0	0	8554
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15087
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9585
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12720
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12249
9	judge	ollama_remote	glm4:latest	0	0	13782

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104609 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9112f3fb47e3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9112f3fb47e3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9112f3fb47e3.tar.gz` (if generated)